



SQL



* Database

- It is a collection of data in a format that can be easily accessed (Digital)
- * Software application used to manage our DB is called DBMS (Database Management System)
- * We use SQL to work with DBMS & we use DBMS to find data ^{from} into Data base

* Types of Databases

* Relational

- Data stored in tables
- * we use SQL to work with relational DBMS (RDBMS)

ex. mySQL

- ORACLE
- Microsoft SQL Server
- PostgreSQL

* Non-Relational (NO SQL)

- Data is not stored in table format
- * we don't use SQL in this

ex. MongoDB

SET SQL_SAFE_UPDATES = 0;

→ to off safe mode

What is SQL

- It is a structured query language
- SQL is a programming language used to interact with relational databases
- It is used to perform CRUD operation

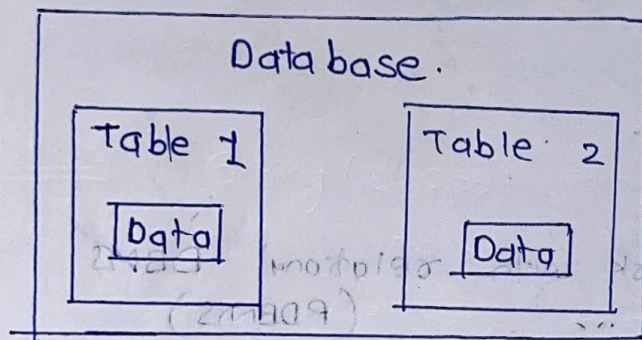
C - Create

R - Read

U - Update

D - Delete

* Data base structure



- Database consists of one or more table inside it
- There is inter-related data inside the table in database

* What is table

Student table

Roll No	Name	class	Gender DoB	Gender DoB	city	Marks
1	Nandya	X	M	1995-06-01	Agao	55
2	Saavabh sonal	XI	M	1995-01-28	mombai	462
3	Trisha	XI	F	1997-12-01	Pune	400
4	Store	X	F	199	Ner	450
5	Maisha	X	M		Delhi	369
6			F		Dubai	250
7	Neha	X	F		NPR	391
8	Nishant	XI	M		Amravati	489

* From column we get to know the general structure (design) / schema of table.

* Rows will tell about data of individual student.

* creating our First Database

- **CREATE DATABASE** db-name; --- To create new database
 (OP)
 create database
- **DROP DATABASE** db-name; --- to delete existing database
- **USE** db-name; --- To use some database to create delete entry for database

* creating our First Table

CREATE TABLE table-name

(column-name 1 datatype constraint,
 column-name 2 datatype constraint,
 column-name 3 datatype constraint);

example :

CREATE TABLE student (
 id INT PRIMARY KEY, → no limit
 name VARCHAR(50),
 age INT NOT NULL);

→
should not be empty

student

id	name	age

* Inserting data into table

↑
(id, name, roll no)
VALUES (1, "Aman", 24);

• INSERT INTO

Table-name

VALUES (2 "Om", 23);

• INSERT INTO

tb-name

VALUES (2 "Om", 23);

* Point table

SELECT * FROM table-name;

* SQL Data-types

* They define the types of value that can be stored in column.

1) Char :- Storing (0-255), can store characters of fixed length

• It takes static memory (inefficient)

CHAR(50)

2) Varchar :- Storing (0-255), can store characters upto given length

* It is most used for storing

VARCHAR(50)

3) BLOB :- Storing (0-65535), can store binary large objects

→ used to store file data.

BLOB(1000)

4. int :: use to store int value.

integer $(-9,223,372,036 - 2^{31}$ to $2^{31})$

INT

5. Tiny int :: small version of int
 $(-128$ to $127)$

TINY INT

6. Big int :: used to store very large integers data
 $(-9^{18}$ to $9^{18})$

BIG INT

7. BIT :: can store x-bit values. x can range from
1 to 64

BIT (2)

→ store 2 bit value
e.g. 00, 01, 10, 11

8. Float :: Decimal no with precision to 23 digits

FLOAT

9. Double :: decimal no with 24 to 53 digits

DOUBLE

10. Boolean :: Boolean values - 0 or 1 ie T or F

BOOLEAN

11. Date :: date in format of YYYY-MM-DD ranging from
1000-01-01 to 9999-12-31

DATE

12. YEAR :: years in 4 digits format ranging from 1901 to 2155

YEAR

* Signed & unsigned datatype

• TINYINT UNSIGNED (0 to 255)

→ store only +ve values

• TINYINT (-128 to 127)

→ store both +ve as well as -ve

* Types of SQL commands

• DDL (Data Definition language):-

create, alter, rename, truncate
& drop

• DQL (Data Query language):-

Select

• DML (Data Manipulation language):-

insert, update & delete

• DCL (Data control language):-

grant & revoke permission to users

• TCL (Transaction control language):-

Start transaction, commit, rollback

* Database related queries

CREATE DATABASE db-name;

✓ CREATE DATABASE IF NOT EXISTS db db_name;

DROP DATABASE db-name;

✓ DROP DATABASE IF ~~NOT~~ EXISTS db-name;

• SHOW DATABASES; --- shows all the DB present in the system

• SHOW TABLES; -- Show all table of the DB which we are using

* Table related queries

* create

→ used to define the schema of the table (design) → which column

ex

CREATE TABLE tb-name (

column_name1 datatype constraint,

column_name2 datatype constraint,

);

ex

CREATE TABLE student (

roll no INT PRIMARY KEY,

name VARCHAR (50)

);

* primary key

→ It should not be null (empty).

→ Should contain unique value for each column

④ select & view all column of table

SELECT * FROM tb-name;

ex SELECT * FROM student;
 ↓
 all

→ Whenever we want to see details of table

* Insert

INSERT INTO tb-name
(column-name1, col-name2)

VALUES

(col1-v1, col2-v1);

(col2, v2, col2 v2);

ex

INSERT INTO student (roll no, name)
VALUES (101, "Atharva"), (102, "BJ");

* practice Q-1

Q- create a database for your company named xyz

• Step1:- create a table inside this DB to store employee info (id, name, & salary).

• Step2:- Add info into DB

1, "adan", 25000

2, "bob", 30000

3, "casey", 40000

Step 3:- select & view all your table data

→ CREATE DATABASE XYZ;

USE XYZ;

```
CREATE TABLE emp(  
  id INT PRIMARY KEY,  
  name VARCHAR(50),  
  salary INT);
```

```
INSERT INTO emp (id, name, salary) VALUES  
(1, 'adam', 25000), (2, 'bob', 30000), (3, 'cathy', 40000);
```

* Keys

* Tables in DB have special column & that special column is known as key

o primary key

→ It is a column (or set of columns) in a table that uniquely identifies each row (a unique id).

• There is only 1 primary key in table & should be NOT NULL

ex emp-id

o Foreign key

A/foreign key is a column (or set of columns) in a table that refers to the primary key of another table.

• There can be multiple FKs

• FKs can have duplicate & null values

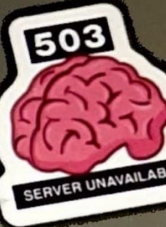


table 1 - Student

id (PK)	name	cityId (FK)	city
101	Karan	1	Pune
102	Aarjun	2	Mumbai
103	Ran	1	Pune
104	Shyam	3	Delhi

table 2 - city

id (PK)	city-name
1	Pune
2	Mumbai
3	Delhi

* There is no compulsion that these should be foreign key there must unique values

* Constraints

SQL constraints are used to specify rules for data in table

① NOT NULL :- columns cannot have a null value

col1 INT NOT NULL

② UNIQUE :- all values in column are different

col1 INT UNIQUE

③ Primary key :- makes a column unique & not null but used only for one.

id INT PRIMARY KEY

(OR)

CREATE TABLE temp (
id int NOT NULL,
PRIMARY KEY (id)

);

- * There is only one primary key in the table
- * if we want to make mixture of two columns primary then

PRIMARY KEY (id, name) $\rightarrow$ mixture of both ^{combination} column will be primary

$\rightarrow$ in this case it is possible that id will duplicate, name will duplicate but the combination of name & id will not be duplicate

* FOREIGN KEY

$\rightarrow$ This table ke primary key dusre table me column reh sakti hai

$\rightarrow$ ie foreign primary key of one table can be present in another table but that will not primary

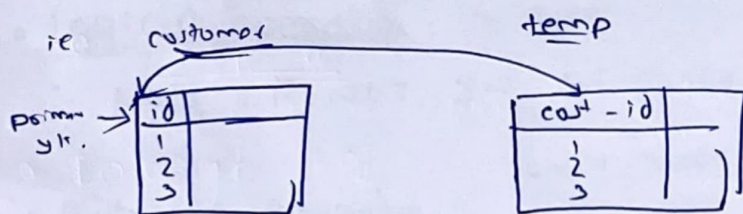
* It is use to link between two tables

EX CREATE TABLE temp (

cust-id INT,

FOREIGN KEY (cust-id) references (customer (id))

another table.



* DEFAULT :- use to sets the default value of a column

Salary INT DEFAULT 25000

EX if in table we don't specify salary after name or id then by default it will take 25,000 & if we change then it will take changed value

* CHECK :-

We can set limit to the values in the column

- Value add krne se Hame check krta deta

ex CREATE TABLE CITY (

id PRIMARY KEY,

city VARCHAR (50),

age INT,

CONSTRAINT random-name CHECK (age $\geq$ 18 AND

city = "DELHI")

);

(OR)

CREATE TABLE newTab

(age INT CHECK (age $\geq$ 18));

* Select

→ Use to select any data from the database

(6) * Basic syntax

SELECT col 1, col 2 FROM table-name;

* To Select all

SELECT * FROM

table-name;

* To check distinct value

SELECT DISTINCT city FROM table-name;

* Where clause

→ We can use different ^{classes} ~~constraints~~ with SELECT constrain

* Where clause is used to define some conditions.

SELECT col1, col2 FROM table-name WHERE condition;

eg. SELECT * FROM student WHERE marks > 80;

* SELECT * FROM student WHERE city = "Mumbai"

* operators in where clause

• Arithmetic operators

:- + (addition), - (subtraction), * (multiplication),
/ (division), % (modulus)

• Comparison operators

:- = (equal to), != (not equal to), >, <=, <, >=

• logical operators

:- AND, OR, NOT, IN, BETWEEN, ALL, LIKE, ANY

• Bitwise operators

:- & (Bitwise AND), | (Bitwise OR)

* operators

• AND (to check for both conditions to be true)

→ SELECT * FROM student WHERE marks > 80
AND city = "Mumbai"

• OR (to check for one of the conditions to be true)

→ SELECT * FROM student WHERE marks > 90
OR city = "Mumbai";

• Between (selects for a given range)

→ SELECT * FROM student WHERE marks
BETWEEN 80 AND 90;

• In (matches any value in the list).

→ SELECT * FROM student WHERE city IN
("Delhi", "Mumbai");

• NOT (to negate the given conditions)

→ SELECT * FROM student WHERE city
NOT IN ("Delhi", "Mumbai");

* LIMIT clause

→ Sets an upper limit on number of (tuples) rows to be returned

SELECT * FROM student LIMIT 3;

SELECT col1, col2 FROM table-name LIMIT numbers;

* Order by clause

→ To sort in ascending (ASC) or descending order (DESC)

SELECT * FROM student ORDER BY City ASC;

SELECT col1, col2 FROM table-name ORDER BY col-name(s) ASC

* Order by Limit

SELECT * FROM student ORDER BY marks DESC LIMIT 3;

* Aggregate Functions

→ It performs a calculation on a set of values & return a single value

• COUNT ()

• MAX ()

• MIN ()

• SUM ()

• AVG ()

SELECT MAX(marks) FROM student;

* Group -By- clause

→ Groups rows that have the same values into Summary rows. It collects data from multiple records & groups, the result by one or more column

* Generally we use group by with some aggregation function

ex. count no of students in each city.

```
SELECT city, count(name) FROM student
```

GROUP BY city;

* Having clause

It is similar to where clause. i.e applies some condition on rows. Used to when we want to apply any condition after grouping.

ex. count no of students in each city where max marks cross 90.

```
SELECT count(name), city FROM student.
```

GROUP BY city.

HAVING max (marks) > 90;

* General order of writing queries

```
SELECT column(s)
FROM table.name
WHERE condition
GROUP BY column(s)
HAVING condition
ORDER BY column(s) ASC;
```

* Where
↓ applies condition on
rows

* Having
↓ applies condition on
groups

ex

```
SELECT city
FROM student
WHERE grade = "A"
GROUP BY city
HAVING MAX(marks) > 90
ORDER BY city DESC;
```

* Table related Queries

o Update (to update existing rows) data

```
UPDATE table-name
SET
```

```
UPDATE table-name
SET col1 = val1, col2 = val2
WHERE condition;
```

ex

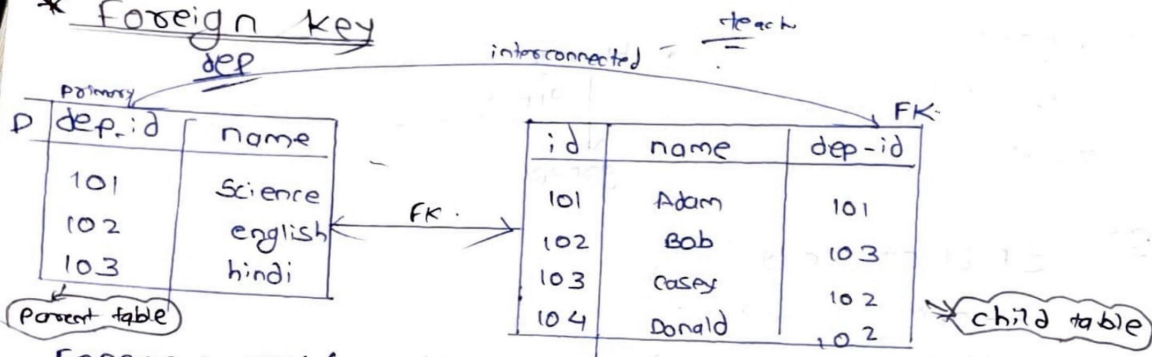
```
UPDATE student
SET grade = "O"
WHERE grade = "A";
```

* Delete (to delete existing rows)

DELETE FROM table-name
WHERE condition;

DELETE FROM student
WHERE marks < 33;

* Foreign key



FOREIGN KEY (dep-id)

REFERENCES ~~dep~~ id dep (pdep-id)

ex

```
CREATE TABLE dep (  
  id INT PRIMARY KEY,  
  name VARCHAR(50)  
);
```

```
CREATE TABLE teacher(  
  id INT PRIMARY KEY,  
  name VARCHAR(50),  
  dep-id INT,
```

```
FOREIGN KEY (dep-id) REFERENCE dep (id)  
);
```

* Cascading for FK

* It ensures that if we update or delete any row in parent table the that row should be updated or deleted in child table.

• on delete cascade

→ When we create a foreign key using this option, it deletes the referencing row in the child table when the referenced row is deleted in the parent table which has a primary key.

• on update cascade

→ When we create a foreign key using UPDATE CASCADE the referencing rows are updated in the child table when the referenced row is updated in the parent table which has a primary key.

```
CREATE TABLE student (  
  id INT PRIMARY KEY,  
  courseID INT,  
  FOREIGN KEY (courseID) REFERENCES course (id)  
  ON DELETE CASCADE  
  ON UPDATE CASCADE  
);
```

* Table related queries

• Alter (to change the schema)
↓
design

(columns, datatype, constraints)

* ① To ADD column

ALTER TABLE table-name

ADD COLUMN column-name datatype constraint;

② To DROP column

ALTER TABLE table-name

DROP COLUMN column-name;

③ RENAME Table

ALTER TABLE table-name

RENAME TO new-table-name;

④ Change column (rename)

ALTER TABLE table-name

CHANGE COLUMN old-name new-name new-datatype
new-constraint;

⑤ Modify column (modify datatype / constraint)

ALTER TABLE table-name

MODIFY col-name new-datatype new-constraint;

⑧ Truncate (to delete ~~the~~ table's data)

TRUNCATE TABLE table_name;

Q. practice Que

Q. In student table

- Change the name of column "name" to "Full-Name".
- Delete all the students who scored marks less than 80.
- Delete the column for grade.

→ ALTER TABLE student

CHANGE column name fullname VARCHAR(50);

DELETE FROM student

WHERE marks < 80;

ALTER TABLE student

DROP COLUMN grade;

* Joins in SQL

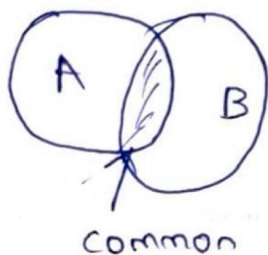
→ Join is used to combine rows from two or more tables, based on a related column between them.

employee		salary	
id	name	id	salary
101		102	
102		103	

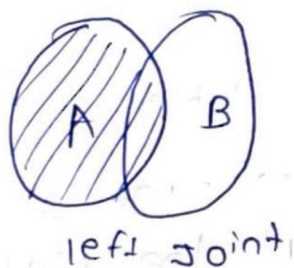
We can join both of table & find the common or specific data that we want

* Types of Joins:

* Inner Join

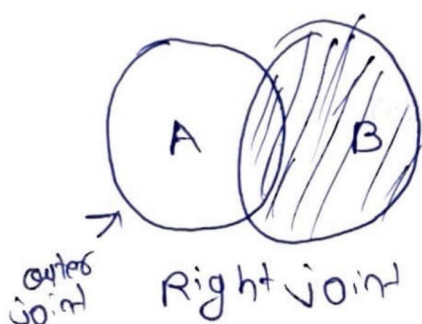


* Left Join

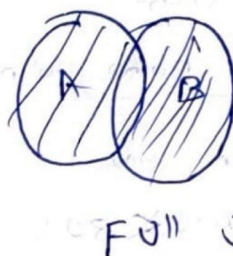


← Outer Join

* Right Join



* Full Join



← Outer Join

* Inner Join

→ Return records that have matching values in both tables

Syntax

SELECT column(s)

FROM table A

INNER JOIN table B

ON table A.col-name = table B.col-name;

ex

student

course

student_id	name

student_id	course

SELECT student

FROM student

INNER JOIN course

ON student.student_id = course.

student_id;

* Left Join

→ Returns all records from the left table, & the matched records from the right table

Syntax

```
SELECT column(s)
FROM table A
LEFT JOIN table B
```

ON table A.col-name = table B.col-name;

```
SELECT *
FROM student as s
LEFT JOIN course as c
ON s.student_id = c.student_id;
```

* Right Join

→ Returns all records from the right table, & the matched records from the left table.

Syntax

```
SELECT column(s)
FROM table A
RIGHT JOIN table B
```

ON table A.col-name = table B.col-name;

* Full Join

→ Returns all records where there is a match in either left or right table

```
SELECT * FROM student as a
LEFT JOIN course as b
ON a.id = b.id
```

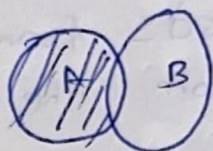
UNION

```
SELECT * FROM student as a
RIGHT JOIN course as b
ON a.id = b.id;
```

RIGHT JOIN

* LEFT EXCLUSIVE JOIN

~~SELECT~~



SELECT *

FROM student as a

LEFT JOIN course as b

ON a.id = b.id

WHERE b.id IS NULL;

* Right Exclusive Join

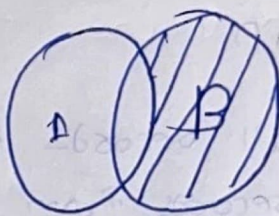
SELECT *

FROM student as a

RIGHT JOIN course as b

ON a.id = b.id

WHERE a.id IS NULL;



* Full Exclusive Join

SELECT *

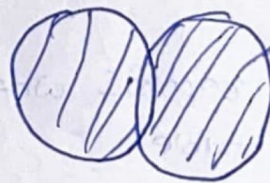
FROM student as a

RIGHT LEFT JOIN course as b

ON a.id = b.id

WHERE b.id IS NULL

UNION



SELECT *

FROM student as a,

RIGHT JOIN course as b

ON a.id = b.id

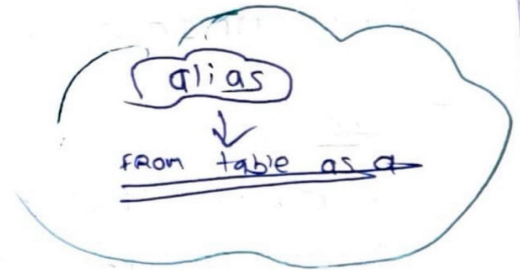
WHERE a.id IS NULL;

* Self Join

→ It is a regular join but the table is join with itself.

Syntax

```
SELECT column(s)
FROM table as a
JOIN table as b
ON a.col-name = b.col-name;
```



ex

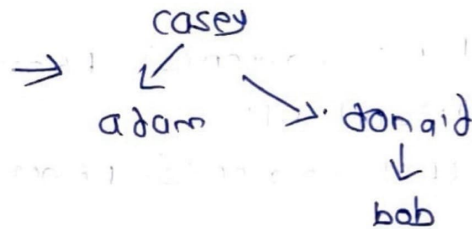
id	name

id	name

both are same table

ex :- Employee

id	name	managers_id
101	adam	103
102	bob	104
103	casey	null
104	donaId	103



→ SELECT *

```
FROM employee as a
JOIN employee as b
ON a.id = b.managers_id;
```

* Union

→ It is used to combine the result set of two or more SELECT statements.
Gives UNIQUE records.

To use it

- every SELECT should have same no. of columns
- columns must have similar data types
- columns in every SELECT should be same order

Syntax

SELECT column(s) FROM table A

UNION

SELECT column(s) FROM table B

gman	61

gman	61

* Union all

→ It allows duplication of ~~the~~ rows or data

ex

SELECT column(s) FROM table A

UNION ALL

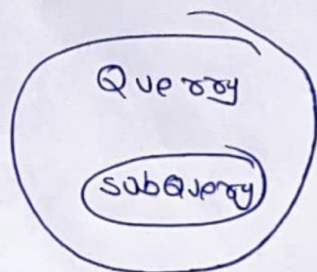
SELECT column(s) FROM table B

61	gman	61
101	gman	101
501	gman	501
103	gman	103
104	gman	104

SQL SUB Queries

A subquery or Inner query or a nested query is a query within another SQL query.

* It involves 2 Select statements.



* We can define sub queries in 3 different ways.

① Inside SELECT

② Inside FROM

③ Inside where

Syntax

SELECT column(s)
FROM table_name
WHERE col_name operator

(subquery):

Ex get names of all students who scored more than class average.

roll_no	name	marks
101	anil	78
102	bharnika	93
103	chetan	85
104	dhruv	96
105	emmanuel	92
106	farrah	82

⇒ Step 1

→ Find avg of class

• Step 2

→ find the names of students with marks > avg

→ SELECT AVG (marks)
FROM student;

SELECT name, marks
FROM student
WHERE marks > 87.6667;

→ Subquery is used to combine both steps

→ SELECT name, marks
FROM student
WHERE marks >

(SELECT AVG (marks) FROM student);

↑
It is dynamic

↑
Subquery

Ex:- Find the names of all students with even roll numbers.

roll no	name	marks
101	anil	78
102	bhumika	93
103	chetan	85
104	dhruv	96
105	emanuel	92
106	faraah	82

- ⇒ ① Find the even roll no
 ② find the names of students with even roll no

⇒ SELECT name, roll no
 FROM student

WHERE roll no IN

(SELECT roll no
 FROM student
 WHERE roll no % 2 = 0)

ex with FROM

Find the max marks from the students of Delhi

roll no	name	marks	city
101	anil	78	Pune
102	bhumika	93	Mumbai
103	chetan	85	Mumbai
104	dhruv	96	Delhi
105	emanuel	92	Delhi
106	faraah	82	Delhi

- ⇒ find the students of Delhi
 ⇒ find their max marks using the subquery in step 2

⇒ SELECT * MAX (marks)

FROM (SELECT * FROM student WHERE city = "Delhi")
 AS temp;

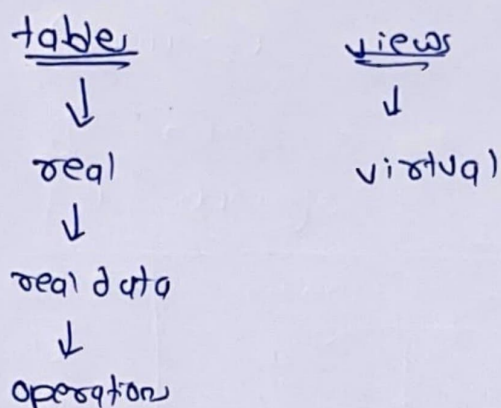
in FROM we need to use alias compulsory

Ex with select

```
SELECT (SELECT MAX(marks) FROM student), name  
FROM student;
```

* MySQL views

→ A view is a virtual table based on the result-set of an SQL statement.



```
CREATE VIEW view1 AS  
SELECT rollno, name FROM student;  
  
SELECT * FROM view1;
```

* A view always shows up-to-date data. The database engine recreates the view, every time a user queries it.

Ex

```
CREATE VIEW view1 view2 AS  
SELECT roll no, name, marks FROM student;
```

```
SELECT * FROM view2  
WHERE marks > 50;
```
